

Estructura de Datos - Ejercicios de Árboles

Alejandro de Jesus Delgado Gonzalez

1. Construcción y Análisis de un Árbol Binario de Búsqueda (ABB)

Descripción: Dada la secuencia de números , se debe describir paso a paso cómo se insertaría cada elemento en un ABB, respetando la propiedad clave: los valores menores al nodo van a la izquierda y los mayores a la derecha.

La clase `NodoArbol` proporciona las funciones auxiliares que ayudan a clasificar un nodo según su propia posición como hijo y el tipo de hijos que tiene el nodo. También mantendrá un seguimiento explícito del padre como un atributo de cada nodo. Usamos parámetros opcionales de python para pasar padres e hijos como parámetros.

Ahora `ArbolBinarioBusqueda` tiene una referencia a la clase `NodoArbol`, si hay nodos en el arbol, la petición se pasa a un método privado definido en la clase `ArbolBinarioBusqueda`. Ahora en la clase hay una metodo (agregar) que nos permite construir nuestro árbol binario de búsqueda. Este método comprobará si el arbol ya tiene una raiz. Si no hay una raiz entonces (agregar) creara un nuevo `NodoArbol` y lo instalara como raiz del arbol. Si ya existe un nodo raiz entonces (agregar) llamara a una funcion auxiliar (`_agregar`) para buscar en el arbol.

Comenzando en la raiz del arbol, buscar la nueva clave con la clave actual del nodo. Si es menor buscar en el sudarbol izquierdo y si es mayor buscar en el sudarbol derecho.

Cuando no hay izquierdo o derecho para buscar, significa que ahi debemos instalar el nuevo nodo, para agregar un nodo al arbol, crear un nuevo objeto en `NodoArbol` e insertar el objeto en el punto descubierto.

Con el método 'agregar' podemos sobrecargar fácilmente el operador '[' para asignación gracias a que hacemos el método '`__setitem__`' llame al metodo 'agregar'. Esto permite escribir instrucciones como un diccionario de Python.

Método `imprimir` hace un recorrido orden inverso lo que ayuda a visualizar la jerarquia.

2. Modelado de un Sistema de Archivos (Árbol N-ario)

Descripción: Se propone diseñar la jerarquía de directorios para un proyecto de software. La raíz es la carpeta **Proyecto**. Dentro de ella, existen tres carpetas: **src**, **docs** y **assets**.

- En **src** hay dos archivos: **main.py** y **utils.py**.
- En **docs** hay una subcarpeta llamada **v1** que contiene el archivo **manual.pdf**.
- En **assets** hay tres imágenes. **Tarea:** Describe esta estructura identificando formalmente qué nodos actúan como **nodos internos** (carpetas) y cuáles como **nodos hoja** (archivos), justificando por qué este es un ejemplo de **Árbol N-ario** debido a su flexibilidad en el grado de los nodos.

En este caso tenemos una clase **Nodo** que adaptamos a una lista de hijos.

El método **agregar_hijo** apoya la extensión natural de la estructura sin preocuparnos por la liberación manual de la memoria; para recorridos recursivos, bastará con iterar sobre hijos.

Proyecto es la raíz, con tres carpetas agregadas.

Dentro de **src** esta [main.py](#) y [utils.py](#)

Dentro de **docs** esta carpeta **v1** y contiene **manual.pdf**

Dentro de **assets** están las 3 imágenes.

3. Compresión de Datos con el Algoritmo de Huffman

Descripción: Supongamos que tenemos un mensaje donde la letra "E" aparece 50 veces, la "A" 30 veces y la "Z" solo 2 veces. **Tarea:** Describe el proceso lógico para construir un **Árbol de Huffman** con estos datos. Debes explicar por qué los caracteres más frecuentes ("E" y "A") deben situarse más cerca de la raíz y cómo se asignarían los códigos binarios (0 para la izquierda, 1 para la derecha) para lograr que el archivo resultante ocupe menos espacio.

Tenemos la clase **NodeTree** representa el nodo del árbol de Huffman. El método **'children'** devuelve esa pareja de hijos para facilitar la construcción recursiva.

El método **'huffman_code_tree'** funciona de manera recursiva.

Si el nodo es un carácter, devuelve un diccionario con el carácter y su código binario acumulado

Si el nodo es un objeto **NodeTree**, obtiene sus hijos y construye los códigos añadiendo 0 para la izquierda y 1 para la derecha

4. Implementación de Autocompletado con un Trie

Descripción: Imagina que estás diseñando el buscador de una tienda en línea y quieres sugerir palabras que comiencen con "cam". En tu diccionario tienes las palabras: "cama", "camino", "campo" y "carro". **Tarea:** Describe cómo se estructuraría un **Trie (árbol de prefijos)** para estas palabras. Debes detallar cómo los nodos comparten la raíz común 'c', 'a' y 'm', y cómo el sistema navegaría por el árbol para listar todos los descendientes del nodo 'm' de forma eficiente en tiempo $O(m)$, superando la eficiencia de una búsqueda lineal.

La clase TrieNode representa cada nodo de Trie. Cada nodo mantiene un diccionario de hijos y una marca que indica si ese nodo corresponde al final de una palabra.

Trie es la clase principal dentro esta formTrie recibe una lista de palabras y las va insertando una por una.

El método insert si no existe caracter como hijo, crea un nuevo nodo al terminar marca al último nodo como final.

Ahora el metodo printAutoSuggestion recorre el Trie siguiendo el prefijo que escribe el usuario.

Si no existe el prefijo, indica que no hay condiciones. Si existe pero no tiene hijos significa que es una palabra sin extensiones.

En otro caso llama a suggestionRec, que recorre los hijos y muestras todas las palabras que cumplen con ese prefijo.

Así Trie aprovecha la búsqueda usando ese prefijo.

5. Análisis Sintáctico de Expresiones (AST)

Descripción: Tomando como base la expresión matemática $(8 + 4) * 2$. **Tarea:** Describe cómo un compilador representaría esta operación mediante un **Árbol de Sintaxis Abstracta (AST)**. Debes identificar cuál sería el nodo raíz (la operación de mayor jerarquía tras evaluar los paréntesis), cuáles serían los nodos internos y cuáles serían las hojas (los literales o números). Explica cómo esta estructura permite al compilador respetar la precedencia de los operadores.

La clase NodoAST es la pieza básica del árbol: cada nodo puede ser un número (hoja) o un operador con dos hijos.

es_hoja: dice si el nodo es solo un número.

evaluar: trabaja de forma recursiva. Si es hoja devuelve el número; si es operador, primero pregunta a sus hijos cuánto valen y luego aplica la operación.

imprimir: dibuja el árbol en texto, mostrando cómo se conectan los nodos. La función construir_ast_simple toma la expresión y arma el árbol: respeta paréntesis y la precedencia de operadores.

Al final, el programa construye el árbol de cada expresión, lo evalúa y lo imprime. Por ejemplo, $(8 + 4) * 2$ se convierte en un árbol con raíz $*$ y da como resultado 24.